

CS336 Assignment 1 (basics) — Writeup

Sections 2-7 (BPE tokenizer, model, training, training loop & generation, experiments) and the experiment log.

Contents

CS336 Assignment 1 — Section 2: BPE Tokenizer (writeup)

- Problem (unicode1): Understanding Unicode (1 pt)
- Problem (unicode2): Unicode Encodings (3 pts)
- Problem (train_bpe): BPE Tokenizer Training (15 pts)
- Problem (train_bpe_tinystories): BPE Training on TinyStories (2 pts)
- Problem (train_bpe_expts_owt): BPE Training on OpenWebText (2 pts)
- Problem (tokenizer): Implementing the tokenizer (15 pts)
- Problem (tokenizer_experiments): Experiments with tokenizers (4 pts)
- Appendix: commands & raw logs

Section 3 — Transformer Language Model Architecture

- Implementation notes
- Problem (transformer_accounting)

Section 4 — Training a Transformer LM

- Problem (learning_rate_tuning)
- Problem (adamw_accounting)

Section 5 - Training loop, Section 6 - Generating text

- 5.1 Data loading (data.py)
- 5.2 Checkpointing (serialization.py)
- 5.3 Training script (train.py)
- 6 Decoding (decoding.py, scripts/generate.py)
- \$7 generate deliverable

Section 7 - Experiments

- Setup and compute accounting
- experiment_log
- learning_rate
- generate
- layer_norm_ablation
- pre_norm_ablation
- no_pos_emb
- swiglu_ablation
- batch_size_experiment
- main_experiment (OpenWebText)
- leaderboard

Experiment log (assignment \$7)

- Runs

CS336 Assignment 1 — Section 2: BPE Tokenizer (writeup)

Problem (unicode1): Understanding Unicode (1 pt)

- (a) `chr(0)` returns the NUL character `U+0000 ('\\x00')`.
- (b) Its `__repr__()` is the escaped form `'\\x00'`, whereas `print(chr(0))` emits the raw NUL byte, which renders as nothing visible (an empty-looking line).
- (c) Inside a string it is just an ordinary character: `"this is a test" + chr(0) + "string"` has `repr` `'this is a test\\x00string'` and prints as `this is a teststring` (the NUL is invisible but still present, `len` counts it and it survives encode/decode).

Problem (unicode2): Unicode Encodings (3 pts)

- (a) UTF-8 gives a base vocabulary of only 256 byte values, is the dominant encoding of web text, is ASCII-compatible (English text is 1 byte/char, so byte sequences are short), and has no byte-order/endianness issue. UTF-16/UTF-32 pad every ASCII character with zero bytes (`"hello".encode("utf-16")` is 12 bytes incl. BOM, UTF-32 is 24 bytes), so sequences get 2–4× longer and are dominated by `\\x00` bytes, which wastes vocabulary/model capacity.
- (b) The function decodes each byte independently, but UTF-8 encodes non-ASCII code points as multi-byte sequences whose individual bytes are not valid on their own. Example: `"é".encode("utf-8") == b'\\xc3\\xa9'`; `decode_utf8_bytes_to_str_wrong(b'\\xc3\\xa9')` raises `UnicodeDecodeError` (`bytes([0xc3]).decode("utf-8")` fails) instead of returning `"é"`. Same for `"牛".encode("utf-8") == b'\\xe7\\x89\\x9b'`.
- (c) `b'\\xc3\\x28'` — `0xc3` announces a 2-byte sequence, but `0x28 (()` is not a continuation byte (10xxxxxx), so the pair is invalid UTF-8. (`b'\\x80\\x80'` also works: lone continuation bytes.)

Problem (train_bpe): BPE Tokenizer Training (15 pts)

Implementation: `cs336_basics/train_bpe.py` (+ `cs336_basics/pretokenization.py`).

- Vocabulary = special tokens + 256 bytes; merges until `vocab_size`.
- Pre-tokenization: file is chunked at `<|endoftext|>` boundaries (`find_chunk_boundaries`), chunks are pre-tokenized in a `multiprocessing.Pool`; inside a chunk the text is first `re.split` on the (escaped) special tokens so no merge crosses a document boundary and special tokens are never counted; each segment is scanned with the GPT-2 regex via `finditer`.
- Merge loop is incremental: `pair_counts`, `pair_to_words` (inverted index) and a lazy-deletion max-heap keyed on `(count, (bytes_a, bytes_b))` so ties break toward the lexicographically greater pair; each merge only rewrites the words that contain the pair and only re-pushes pairs whose counts changed.

Tests: `pytest tests/test_train_bpe.py` — see test log in the appendix.

Problem (train_bpe_tinystories): BPE Training on TinyStories (2 pts)

Run: `scripts/train_bpe_cli.py --input data/TinyStoriesV2-GPT4-train.txt --vocab-size 10000 --out-dir out/ts10k --profile` (2.23 GB, 2 717 699 documents) on a shared 20-core box, pinned to **4 cores** (taskset -c 0-3, nice 15), Python 3.13.

- (a) `train_bpe` took **217 s** (3 min 37 s wall including start-up and serialization); peak memory **716 MB** (`/usr/bin/time -v` max RSS: 109 MB in the main process, 700 MB in the largest pre-tokenization worker). The longest token is the 15-byte `b' accomplishment'`; the next longest are `' disappointment'`, `' responsibility'`, `' uncomfortable'`, `' compassionate'`, `' understanding'`, `' neighbourhood'` — all ordinary long English words with a leading space, i.e. exactly what a whole-word BPE on children's stories should learn (no junk, no punctuation runs). Yes, this makes sense.

(b) cProfile: `count_pretokens_in_file` (pre-tokenization) = **213 s of 217 s (98 %)**; the incremental merge loop for 9 743 merges is only ~4 s because TinyStories has few distinct pre-tokens. Pre-tokenization is a Python-level loop over ~500 M regex matches, so it is CPU-bound and scales with the number of worker processes (we were capped at 4; with all 20 cores it would be well under a minute). After this run we replaced the per-match `finditer` loop by `Counter.update(pattern.findall(doc))` (both run in C) — the 5 M-byte sample went from 0.9 s to 0.3 s and the outputs are byte-identical.

Problem (train_bpe_expts_owt): BPE Training on OpenWebText (2 pts)

Run: `scripts/train_bpe_cli.py --input data/owt_train.txt --vocab-size 32000 --out-dir out/owt32k --profile` on the full 11.9 GB `owt_train.txt` (2 355 962 documents), same 4-core pin. (A 1 GB-prefix dry run took 188 s / 2.1 GB peak, which is how we decided the full run was affordable.)

(a) `train_bpe` took **1 523 s (25.4 min)**; peak memory **9.5 GB** (max RSS of the main process — the merge-loop state: every distinct pre-token as a `list[int]`, the pair → count table, the pair → word inverted index and the lazy heap). Profile: pre-tokenization 460 s (30 %), merge loop ~1 050 s (70 %) — the opposite split from TinyStories, because OWT has far more distinct pre-tokens (the main process holds 9.5 GB of state vs 0.1 GB) and 3× more merges, and each early merge touches millions of words. The longest token (64 bytes) is `b'\xc3\x83\xc3\x82' * 16 = ÃÃÃÃÃ...`, i.e. mojibake — the residue of text that was UTF-8-encoded more than once — which appears verbatim as long runs in many scraped pages; the next longest are `'-' * 64`, `'_' * 16`, `'_' * 32`, `'=' * 32`, `'.' * 32`, `'*' * 32` (ASCII-art separators). This is *not* a linguistically meaningful token, but it does make sense for byte-level BPE on web text: these long identical byte runs occur thousands of times and every merge inside them halves their length, so they win the frequency race all the way to 64 bytes. The longest *alphabetic* tokens are `'disproportionately'`, `'telecommunications'`, `'environmentalists'`, `'unconstitutional'`, `'cryptocurrencies'`, `'counterterrorism'`.

(b) TinyStories (10 K) vs OpenWebText (32 K) vocabularies:

- Overlap: 7 319 tokens are in both — 73 % of the TinyStories vocab, 23 % of the OWT vocab. Almost every TS token that OWT lacks is a children's-story noun/verb (`'granddaughter'`, `'marshmallows'`, `'caterpillars'`, `'veterinarian'`, `'storekeeper'`, `'superheroes'`, `'blueberries'`).
- Both are ~95 % whole words with a leading space (TS 97 %, OWT 95 %); OWT has ~10× more digit-containing tokens (582 vs 16 — years, prices, dates) and 4× more non-ASCII tokens (559 vs 148: curly quotes, `–`, `…`, mojibake).
- OWT contains web/news-specific tokens that TS never sees: `'https'`, `'http'`, `'www'`, `'html'`, `'href'`, `'Advertisement'`, `'Copyright'`, `'Comments'`, `'Share'`, `'Click'`, `'Getty'`, `'Reuters'`, `'Twitter'`, `'Facebook'`, `'Google'`, `'realDonaldTrump'`, `'Obamacare'`, `'#####'`.
- TS long tokens are all emotional/descriptive vocabulary of stories (`'accomplishment'`, `'disappointment'`, `'compassionate'`); OWT long tokens are political/technical (`'unconstitutional'`, `'telecommunications'`) plus the separator/mojibake runs above.

Problem (tokenizer): Implementing the tokenizer (15 pts)

Implementation: `cs336_basics/tokenizer.py` (`Tokenizer.__init__` / `from_files` / `encode` / `encode_iterable` / `decode`).

- `encode`: split on special tokens (longest first, capturing group so the specials are kept), pre-tokenize each remaining segment, and apply merges to each pre-token by repeatedly merging the adjacent pair with the lowest merge rank; per-pre-token cache.
- `encode_iterable`: keeps a small text buffer and only emits the prefix that can no longer change: it holds back the last pre-token, the last `max(len(special))-1` characters (possible special-token prefix) and any trailing whitespace-only pre-tokens (the `\s+(?!\\S)` rule looks ahead), so the stream tokenization equals the one-shot tokenization with O(line) memory.
- `decode`: concatenate bytes, `decode("utf-8", errors="replace")` → U+FFFD for bad bytes.

Problem (tokenizer_experiments): Experiments with tokenizers (4 pts)

All numbers from `scripts/tokenizer_experiments.py` (ratio samples 10 documents with `random.Random(seed)` from the first 200 MB of the validation file; throughput encodes a whole file with `Tokenizer.encode`; `encode` writes a `uint16 .npy`).

(a) Compression ratio (bytes / token) on 10 sampled documents:

tokenizer	data	seed 0	seed 1
TinyStories 10 K	TinyStories valid	4.10 (7 633 B → 1 861 tok)	4.05
OpenWebText 32 K	OpenWebText valid	4.52 (30 001 B → 6 642 tok)	4.43

Over the *whole* validation sets (from (d)): TS 4.12 B/token, OWT 4.37 B/token. So each tokenizer gets ~4.1–4.5 bytes per token on its own domain; the 32 K vocabulary buys a slightly better ratio on the much more diverse web text.

(b) TinyStories tokenizer on OpenWebText documents: **3.27 B/token** (seed 0; 3.43 for seed 1) vs 4.52 with the OWT tokenizer — 30–40 % more tokens for the same bytes. Qualitatively the text gets chopped into sub-word pieces: e.g. ' New Jersey judge ... official misconduct, issuing a criminal summons' → TS: ' Ne' 'w' ' J' 'er' 'sey', ' offic' 'ial', ' mis' 'cond' 'uct', ' is' 's' 'uing', ' cr' 'im' 'inal', ' s' 'umm' 'ons' (87 tokens for 300 B), while the OWT tokenizer keeps ' Jersey', ' official', ' misconduct', ' issuing', ' criminal' whole (60 tokens). On the 10 OWT docs 21 % of the TS tokenizer's output tokens are single bytes and 40 % are ≤ 2 bytes, vs 15 % / 23 % for the OWT tokenizer. (The reverse direction is much milder: OWT tokenizer on TS docs = 3.96 B/token, since the larger web vocabulary already covers most children's-story words.)

(c) Throughput of `Tokenizer.encode` (single process, one pinned core, Python 3.13):

tokenizer	text	throughput
TS 10 K	tinystories_sample_5M.txt (5.2 MB)	5.4 MB/s
TS 10 K	TS valid (22.5 MB)	7.3 MB/s
OWT 32 K	OWT valid, first 50 MB	3.3 MB/s

TinyStories is highly repetitive so the per-pre-token cache hits most of the time; on web text the tokenizer runs at ~3.3 MB/s. At 3.3–7 MB/s the 825 GB Pile would take **~33–70 h single-process** (≈ 2–4 h on 20 cores with the same document-level chunking we use for `encode --num-processes`).

(d) Encoded datasets (uint16 `.npy`, verified with `np.load(mmap_mode="r"): max(id) < vocab_size, <|endoftext|>` count equals the count in the text file):

file	tokens	bytes/token	wall time
ts_valid_10k.npy	5 465 883	4.12	5 s (<code>encode_iterable</code> , 64 MB RSS)
ts_train_10k.npy	541 229 347	4.12	700 s (<code>encode_iterable</code> , 1 core, 196 MB RSS)
owt_valid_32k.npy	66 401 098	4.37	57 s (4 processes)
owt_train_32k.npy	2 727 120 452	4.37	1 848 s (4 processes)

The 2.2 GB TinyStories train set was encoded with the streaming `encode_iterable` path in constant memory (~200 MB RSS); the 12 GB OWT train set with 4 worker processes that each encode a chunk delimited at `<|endoftext|>` boundaries (identical output to one-shot `encode`, checked on the 5 M sample). uint16 is appropriate because vocabulary sizes of 10 000 and 32 000 are both $< 2^{16} = 65\,536$, so every id fits in 2 bytes (half of int32) with no loss; the token arrays are 1.0 GB (TS) and 5.5 GB (OWT) instead of 2× that as int32.

Appendix: commands & raw logs

Environment: C12 shared box, Python 3.13.15 (uv-managed), torch 2.11.0+cu130, `regex`, `tiktoken`; every job pinned with `nice -n 15 taskset -c 0-3, OMP_NUM_THREADS=4`. Full logs are in `/home/c12/workspace/claude_workspace/cs336_a1/logs/`.

```

$ .venv/bin/python -m pytest tests/test_train_bpe.py tests/test_tokenizer.py -q
===== 27 passed, 1 xpassed in 3.57s =====
# (test_encode_memory_usage is marked xfail; it happens to pass here)

$ /usr/bin/time -v .venv/bin/python scripts/train_bpe_cli.py --input data/TinyStoriesV2-GPT4-train.txt \
--vocab-size 10000 --out-dir out/ts10k --profile
vocab_size=10000 merges=9743
time=217.2s peak_rss(main)=109MB peak_rss(max child)=700MB
longest token (15 bytes): b' accomplishment'
1 0.000 0.000 213.310 213.310
cs336_basics/pretokenization.py:118(count_pretokens_in_file)
Elapsed (wall clock) time (h:mm:ss or m:ss): 3:37.23
Maximum resident set size (kbytes): 716352

$ /usr/bin/time -v .venv/bin/python scripts/train_bpe_cli.py --input owt_train_1G.txt --vocab-size 32000
... # 1 GB dry run
time=188.1s peak_rss(main)=2059MB peak_rss(max child)=436MB
Maximum resident set size (kbytes): 2108032

$ ulimit -v 22000000; /usr/bin/time -v .venv/bin/python scripts/train_bpe_cli.py --input
data/owt_train.txt \
--vocab-size 32000 --out-dir out/owt32k --profile
vocab_size=32000 merges=31743
time=1523.0s peak_rss(main)=9302MB peak_rss(max child)=473MB
longest token (64 bytes): b'\xc3\x83\xc3\x82\xc3\x83\xc3\x82...' (x16)
1 428.950 428.950 1513.812 1513.812 cs336_basics/train_bpe.py:68(train_bpe)
1 0.000 0.000 459.649 459.649
cs336_basics/pretokenization.py:122(count_pretokens_in_file)
Elapsed (wall clock) time (h:mm:ss or m:ss): 25:23.44
Maximum resident set size (kbytes): 9525604

$ scripts/tokenizer_experiments.py ratio --vocab out/ts10k/vocab.json --merges out/ts10k/merges.txt --
input data/TinyStoriesV2-GPT4-valid.txt
10 docs, 7633 bytes, 1861 tokens -> 4.102 bytes/token
$ ... ratio (TS10k) --input data/owt_valid.txt
10 docs, 30001 bytes, 9167 tokens -> 3.273 bytes/token
$ ... ratio (OWT32k) --input data/owt_valid.txt
10 docs, 30001 bytes, 6642 tokens -> 4.517 bytes/token
$ ... ratio (OWT32k) --input data/TinyStoriesV2-GPT4-valid.txt
10 docs, 7633 bytes, 1927 tokens -> 3.961 bytes/token

$ ... throughput (TS10k) --input tests/fixtures/tinystories_sample_5M.txt
5242880 bytes -> 1274107 tokens in 0.98s = 5.36 MB/s ; Pile (825GB) would take ~42.7 h single-process
$ ... throughput (TS10k) --input data/TinyStoriesV2-GPT4-valid.txt
22502601 bytes -> 5465883 tokens in 3.06s = 7.34 MB/s ; Pile ~31.2 h
$ ... throughput (OWT32k) --input owt_valid_50M.txt
50000000 bytes -> 11458749 tokens in 15.30s = 3.27 MB/s ; Pile ~70.1 h

$ ... encode (TS10k) --input data/TinyStoriesV2-GPT4-valid.txt --output data/ts_valid_10k.npy
wrote 5465883 tokens (uint16) to data/ts_valid_10k.npy in 5s (max RSS 63652 KB)
$ ... encode (TS10k) --input data/TinyStoriesV2-GPT4-train.txt --output data/ts_train_10k.npy
wrote 541229347 tokens (uint16) to data/ts_train_10k.npy in 700s (max RSS 195932 KB)
$ ... encode (OWT32k) --input data/owt_valid.txt --output data/owt_valid_32k.npy --num-processes 4 --num-
chunks 128
wrote 66401098 tokens (uint16) to data/owt_valid_32k.npy in 56s (max RSS 183900 KB)
$ ... encode (OWT32k) --input data/owt_train.txt --output data/owt_train_32k.npy --num-processes 4 --num-
chunks 1024
wrote 2727120452 tokens (uint16) to data/owt_train_32k.npy in 1848s (max RSS 206748 KB, wall 30:49)

$ python -c "np.load(f, mmap_mode='r')" # sanity
data/ts_valid_10k.npy uint16 (5465883,) max 9999 < 10000 eot_count 27630
data/ts_train_10k.npy uint16 (541229347,) max 9999 < 10000 eot_count 2717699
data/owt_valid_32k.npy uint16 (66401098,) max 31999 < 32000 eot_count 59059
data/owt_train_32k.npy uint16 (2727120452,) max 31999 < 32000 eot_count 2399397

```

Section 3 — Transformer Language Model Architecture

Implementation: `cs336_basics/model.py` (Linear, Embedding, RMSNorm, silu/SwiGLU, RotaryPositionalEmbedding, softmax, scaled_dot_product_attention, MultiHeadSelfAttention, TransformerBlock, TransformerLM). Tests: `tests/test_model.py` (13) + `test_softmax_matches_pytorch` — 14 passed. Accounting numbers below are produced by `scripts/flops_accounting.py` (`.venv/bin/python scripts/flops_accounting.py --md`); the GPT-2 XL parameter count was also cross-checked against `sum(p.numel() for p in TransformerLM(...).parameters())` on a meta device.

Implementation notes

- **Linear** stores $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ and computes `einsum(x, W, "... d_in, d_out d_in -> ... d_out")`; init `trunc_normal(0,  $\sigma^2 = 2/(d_{\text{in}}+d_{\text{out}})$ )` clipped at $\pm 3\sigma$. **Embedding** $N(0,1)$ clipped at ± 3 , lookup by advanced indexing. **RMSNorm** upcasts to fp32, `x / sqrt(mean(x2)+eps) * g`, casts back.
- **SwiGLU** = $W_2 (\text{SiLU}(W_1 x)) \odot W_3 x$; `d_ff` defaults to `8/3·d_model` rounded to a multiple of 64.
- **RoPE** rotates adjacent pairs $(x_{\{2k-1\}}, x_{\{2k\}})$ by $\theta_{\{i,k\}} = i \cdot \theta^{\{-(2k-2)/d\}}$; `cos/sin` tables of shape `(max_seq_len, d_k/2)` are non-persistent buffers, indexed by `token_positions`; one module is shared by all layers of the LM. Missing leading dims of `token_positions` (e.g. the head dim) are broadcast.
- **softmax** subtracts the max along `dim`. **SDPA** builds scores with `einsum` over arbitrary leading dims, `masked_fill(~mask, -inf)`, `softmax`, `einsum` with `V`. **MHSA** does one matmul each for `Q/K/V` (all heads at once), splits heads into a batch dim with `rearrange`, applies RoPE to `Q` and `K` only, uses a `tril` causal mask.
- **TransformerBlock**: `x + Attn(RMSNorm(x))`, then `x + FFN(RMSNorm(x))`. **TransformerLM**: embedding $\rightarrow$ blocks $\rightarrow$ `ln_final` $\rightarrow$ untied `lm_head` $\rightarrow$ logits. Parameter names match the reference state dict (`token_embeddings`, `layers.i.attn.{q,k,v,output}_proj`, `ln1/ln2`, `ffn.w1/w2/w3`, `ln_final`, `lm_head`), so the adapters call `load_state_dict` directly.
- **Section-7 ablation switches** on `TransformerBlock` / `TransformerLM` (defaults = standard architecture, the 14 tests above still pass): `norm="rmsnorm"|"none"`, `norm_position="pre"|"post"` (post: `z = Norm(x + MHA(x))`; `y = Norm(z + FFN(z))`), and `ln_final` is dropped since every block already ends with a norm), `use_rope=True|False` (False = NoPE), `ffn_type="swiglu"|"silu"` (`FFN(x) = W2 SiLU(W1 x)`; `d_ff` defaults to `4·d_model` when not given).

Problem (transformer_accounting)

Architecture as implemented in this assignment: no biases, SwiGLU with three matrices, RoPE has no parameters, RMSNorm has one gain vector, LM head not tied to the embedding. Only matmuls are counted for FLOPs, using the rule $(m \times n) @ (n \times p) = 2mnp$.

(a) GPT-2 XL parameters and memory

Per layer: $4 \cdot d^2$ (`Q,K,V,O`) + $3 \cdot d \cdot d_{\text{ff}}$ (`W1,W2,W3`) + $2 \cdot d$ (two RMSNorm gains) = $10,240,000 + 20,582,400 + 3,200 = 30,825,600$. Plus token embedding $V \cdot d = 80,411,200$, `ln_final` 1,600, `lm_head` $V \cdot d = 80,411,200$.

Total = $48 \cdot 30,825,600 + 2 \cdot 80,411,200 + 1,600 = 1,640,452,800 \approx 1.64 \text{ B parameters}$. In fp32 that is $4 \times 1,640,452,800 = 6,561,811,200$ bytes $\approx 6.56 \text{ GB}$ (6.11 GiB) just to hold the weights (no gradients/optimizer state/activations).

(b) Matrix multiplies in one forward pass ($T = 1024$ tokens)

matmul	shape	FLOPs	share
attn: QKV projections	48 layers $\times$ 3 $\times$ [(1024 $\times$ 1600)@(1600 $\times$ 1600)]	7.550e+11	21.5%
attn: Q K ^T scores	48 layers $\times$ 25 heads $\times$ [(1024 $\times$ 64)@(64 $\times$ 1024)]	1.611e+11	4.6%
attn: P V	48 layers $\times$ 25 heads $\times$ [(1024 $\times$ 1024)@(1024 $\times$ 64)]	1.611e+11	4.6%
attn: output projection	48 layers $\times$ (1024 $\times$ 1600)@(1600 $\times$ 1600)	2.517e+11	7.2%
ffn: W1 x and W3 x	48 layers $\times$ 2 $\times$ [(1024 $\times$ 1600)@(1600 $\times$ 4288)]	1.349e+12	38.4%
ffn: W2 (·)	48 layers $\times$ (1024 $\times$ 4288)@(4288 $\times$ 1600)	6.744e+11	19.2%
lm_head	(1024 $\times$ 1600)@(1600 $\times$ 50257)	1.647e+11	4.7%

Closed form: per layer $2T \cdot (4d^2 + 3d \cdot d_{\text{ff}}) + 4T^2d$, plus $2T \cdot d \cdot V$ for the LM head.

Total $\approx 3.517 \times 10^{12}$ FLOPs (3,516,769,894,400) per forward pass.

(c) Most expensive parts

The SwiGLU feed-forward network dominates (57.5 % of all FLOPs; W1/W3 alone are 38 %), followed by the attention Q/K/V/output projections (28.6 %). The actual attention score computation (QK^T and PV) is only 9.2 % at context 1024, and the LM head 4.7 %.

(d) Other GPT-2 sizes

$d_{\text{ff}} = 8/3 \cdot d_{\text{model}}$ rounded to a multiple of 64: small 2048, medium 2752, large 3392, XL 4288.

model	params	total FLOPs	attn projections (QKV+out)	attn scores (QK ^s +PV)	feed-forward (SwiGLU)	lm_head
GPT-2 small (12L, 768, 12h)	0.162 B	2.916e+11	19.9%	13.3%	39.8%	27.1%
GPT-2 medium (24L, 1024, 16h)	0.407 B	8.302e+11	24.8%	12.4%	50.1%	12.7%
GPT-2 large (36L, 1280, 20h)	0.834 B	1.769e+12	27.3%	10.9%	54.3%	7.4%
GPT-2 XL (48L, 1600, 25h)	1.640 B	3.517e+12	28.6%	9.2%	57.5%	4.7%

Per-matmul breakdowns for every model are printed by the script. As the model grows, the layer-internal matmuls that scale as $L \cdot d^2$ (FFN and attention projections) take a proportionally larger share, while the LM head ($d \cdot V$, one copy, grows only linearly in d) drops from 27 % to under 5 %, and the attention-score matmuls ($L \cdot T^2 \cdot d$, linear in d) shrink from 13 % to 9 % because at fixed $T=1024$ they grow slower than the d^2 terms.

(e) GPT-2 XL with context length 16,384

Total forward FLOPs go from $3.52e^{12}$ to $1.336e^{14}$ — a $38\times$ increase for a $16\times$ longer sequence. Every matmul except QK^T/PV is linear in T ($16\times$), whereas the attention-score matmuls are quadratic ($256\times$), so they go from 9.2 % to 61.7 % of the total and become the dominant cost; the FFN drops to 24.2 %, projections to 12.1 %, LM head to 2.0 %.

Section 4 — Training a Transformer LM

Code: `cs336_basics/nn_utils.py` (`cross_entropy`, `gradient_clipping`), `cs336_basics/optimizer.py` (SGD, AdamW, `get_lr_cosine_schedule`). Scripts: `scripts/sgd_lr_toy.py`, `scripts/adamw_accounting.py`.

Problem (learning_rate_tuning)

Ran the handout's toy loop (`weights = Parameter(5*randn(10,10))`, `loss = (weights**2).mean()`, decaying SGD `lr/sqrt(t+1)`) for 10 iterations with a fixed seed (`scripts/sgd_lr_toy.py`):

lr	loss at t = 0 ... 9
1e0 (baseline)	26.27, 25.23, 24.52, 23.96, 23.48, 23.06, 22.69, 22.35, 22.03, 21.74
1e1	26.27, 16.81, 12.39, 9.697, 7.855, 6.513, 5.492, 4.693, 4.053, 3.531
1e2	26.27, 26.27, 4.507, 0.1079, 1.1e-16, 1.2e-18, 4.1e-20, 2.5e-21, 2.1e-22, 2.4e-23
1e3	26.27, 9484, 1.64e6, 1.82e8, 1.48e10, 9.32e11, 4.78e13, 2.06e15, 7.58e16, 2.44e18

Answer. `lr=1e1` decays clearly faster than `lr=1` ($26 \rightarrow 3.5$ vs $26 \rightarrow 21.7$ in 10 steps). `lr=1e2` overshoots on the first step (the update $\theta \leftarrow \theta - 100 \cdot (20/100) = -\theta$ leaves the loss unchanged) and then decays *extremely* fast to $\sim 1e-23$ once the $1/\sqrt{t+1}$ decay shrinks the effective step below 2. `lr=1e3` diverges: the loss grows by $\sim$ two orders of magnitude per step (each update multiplies θ by $1 - 20/\sqrt{t+1}$, whose magnitude stays > 1 for all 10 steps).

Problem (adamw_accounting)

Notation: B = batch_size, L = context_length, N = num_layers, d = d_{model} , h = num_heads, V = vocab_size, $d_{\text{ff}} = 8/3 \cdot d$. Every tensor is float32 (4 bytes). Numbers computed by `scripts/adamw_accounting.py`.

(a) Peak memory

Parameters (untied token embedding and LM head, RMSNorm gains, no biases, RoPE has no parameters):

- embedding $V \cdot d$; per layer $4d^2$ (Q,K,V,O) + $3 \cdot d \cdot d_{\text{ff}} = 8d^2$ ($W1, W2, W3$) + $2d$ (two RMSNorm gains); final RMSNorm d ; LM head $d \cdot V$.
- $P = 2Vd + N(12d^2 + 2d) + d$ floats $\rightarrow$ **4P bytes**.

Activations (only the components listed in the problem, counted as floats saved for backward, per batch element and then $\times B$):

- Transformer block, per layer:
 - RMSNorms: $2 \times L \cdot d$
 - Attention: QKV projections $3 \cdot L \cdot d$; QK^T scores $h \cdot L^2$; softmax $h \cdot L^2$; weighted sum of values $L \cdot d$; output projection $L \cdot d \rightarrow 5Ld + 2hL^2$
 - SwiGLU FFN: $W1 \times L \cdot d_{\text{ff}}$; $W3 \times L \cdot d_{\text{ff}}$; SiLU $L \cdot d_{\text{ff}}$; element-wise product $L \cdot d_{\text{ff}}$; $W2$ output $L \cdot d \rightarrow 4L \cdot d_{\text{ff}} + Ld = (35/3)Ld$
 - per layer: $(2 + 5 + 35/3) \cdot L \cdot d + 2hL^2 = (56/3) \cdot L \cdot d + 2hL^2$
- final RMSNorm $L \cdot d$; output embedding (logits) $L \cdot V$; cross-entropy on logits (log-softmax / probabilities kept for backward) $L \cdot V$.
- $A = B \cdot [N \cdot ((56/3) \cdot L \cdot d + 2hL^2) + L \cdot d + 2L \cdot V]$ floats $\rightarrow$ **4A bytes**.

Gradients: one float per parameter $\rightarrow$ **4P bytes**. **Optimizer state:** AdamW keeps m and v , each the size of the parameters $\rightarrow$ **8P bytes**.

Total peak memory (bytes): $M = 4 \cdot (P + A + P + 2P) = 4 \cdot (4P + A) = 16 \cdot [2Vd + N(12d^2 + 2d) + d] + 4B \cdot [N((56/3)Ld + 2hL^2) + Ld + 2LV]$.

(b) GPT-2 XL ($V=50257$, $L=1024$, $N=48$, $d=1600$, $h=25$, $d_{\text{ff}}=8/3 \cdot d$)

- $P = 1.6355e9$ (1.64 B parameters); $4P$ in fp32 = 26.17 GB for params+grads+m+v.

- Activations per example: $A/B = 4.089e9$ floats = 16.36 GB.
- **memory** $\approx 16.36 \text{ GB} \cdot \text{batch_size} + 26.17 \text{ GB}$ ($1.6357e10 \cdot B + 2.6169e10$ bytes; in GiB: $15.23 \cdot B + 24.37$).
- 80 GB budget: $B \leq (80 - 26.17)/16.36 = 3.29 \rightarrow$ **maximum batch size = 3** (B=3 uses 75.2 GB, B=4 needs 91.6 GB; the same B=3 holds if "80 GB" means 80 GiB).

The activation term dominates: with fp32 activations and no activation checkpointing, GPT-2 XL barely fits three 1024-token sequences on an 80 GB card even though the static state is only ~26 GB.

(c) FLOPs of one AdamW step

AdamW is purely element-wise over the parameter vector, so cost is $\mathcal{O}(P)$. Per parameter, Algorithm 1 does: weight decay $\theta \leftarrow \theta - \alpha \lambda \theta$ (2 ops), $m \leftarrow \beta_1 m + (1-\beta_1)g$ (3), $v \leftarrow \beta_2 v + (1-\beta_2)g^2$ (4), and $\theta \leftarrow \theta - \alpha_t \cdot m / (\sqrt{v} + \epsilon)$ (5: sqrt, add, div, mul, sub); α_t is a scalar computed once. So roughly **$\approx 14 \cdot P$ FLOPs per step** ($\approx 2.3e10$ for GPT-2 XL) — a small constant times P , i.e. negligible next to the $\approx 6 \cdot P \cdot (B \cdot L)$ FLOPs of forward+backward, since it does not scale with the number of tokens.

(d) Training time for GPT-2 XL, 400K steps, batch 1024, one H100 at 50% MFU

Forward matmul FLOPs per token (2 FLOPs per multiply-add):

- per layer: QKV $6d^2$, QK^T $2Ld$, AV $2Ld$, output proj $2d^2$, SwiGLU $3 \cdot 2 \cdot d \cdot d_{ff} = 16d^2 \rightarrow 24d^2 + 4Ld$
- LM head $2dV$
- $F_{\text{fwd}}/\text{token} = N(24d^2 + 4Ld) + 2dV = 3.4245e9$ FLOPs for GPT-2 XL.

Per step: tokens = $1024 \times 1024 = 1.05e6$; forward+backward = $3 \times F_{\text{fwd}}$ (backward = $2 \times$ forward) $\rightarrow 3 \times 3.4245e9 \times 1.0486e6 = 1.077e16$ FLOPs (+ $1.4e10 \cdot \dots \approx 2.3e10$ for AdamW, negligible). Total for 400K steps: $4.31e21$ FLOPs. Sustained throughput at 50% MFU: $0.5 \times 495e12 = 2.475e14$ FLOP/s. time = $4.31e21 / 2.475e14 = 1.74e7 \text{ s} \approx 4,840 \text{ hours} \approx 202 \text{ days}$ on a single H100.

(Non-matmul FLOPs — softmax, RMSNorm, SiLU, RoPE — are ignored, as is standard for this estimate.)

Section 5 - Training loop, Section 6 - Generating text

Code: `cs336_basics/data.py`, `cs336_basics/serialization.py`, `cs336_basics/train.py` (training CLI, run as `python -m cs336_basics.train`), `cs336_basics/decoding.py`, `scripts/generate.py`. Tests: `tests/test_data.py`, `tests/test_serialization.py` (both pass: 2 passed).

5.1 Data loading (`data.py`)

`get_batch(dataset, batch_size, context_length, device)` draws `batch_size` start indices $i \sim \text{Uniform}\{0, \dots, n - m - 1\}$ ($n = \text{len}(\text{dataset})$, $m = \text{context_length}$), gathers one $(B, m+1)$ window with a single fancy-index (`dataset[starts[:, None] + arange(m+1)]`, which copies only those rows out of a memmap), and returns $x = \text{window[:, :-1]}$, $y = \text{window[:, 1:]}$ as `int64` tensors moved to device. An optional `numpy.random.Generator` argument makes sampling reproducible (used for the fixed validation batches). `load_token_file(path, dtype=uint16)` opens `.npy` files with `np.load(mmap_mode="r")` and raw files with `np.memmap`, so neither train nor validation set is ever read into RAM; `get_batch` cost is ~ 0.4 ms for $B=16$, $m=64$ (measured on CPU).

5.2 Checkpointing (`serialization.py`)

`save_checkpoint(model, optimizer, iteration, out, extra=None)` writes `{"model": state_dict, "optimizer": state_dict, "iteration": int, "extra": ...}` with `torch.save` (path or file-like). `load_checkpoint(src, model, optimizer)` restores both state dicts and returns `iteration`. Two details: a `torch.compile` model is unwrapped (`_orig_mod`) before `state_dict()` so the keys carry no `_orig_mod.` prefix and load into a plain model; tensors are loaded with `map_location="cpu"` and moved to the parameter device by `load_state_dict`, so a GPU checkpoint loads on CPU and vice-versa. `extra` carries the run config, `elapsed_s` and `tokens_seen`, which is what makes `--resume` continue the wall-clock axis of the loss curve instead of restarting it at 0.

5.3 Training script (`train.py`)

`python -m cs336_basics.train --train-data ... --val-data ... --out-dir RUN [flags]`. Full flag list: `--help` (also reproduced in the report). Design:

- **Model** is `cs336_basics.model.TransformerLM(vocab_size, context_length, d_model, num_layers, num_heads, d_ff, rope_theta)`. \$7 ablation flags map onto the model's knobs: `--no-rmsnorm` $\rightarrow$ `norm="none"`, `--post-norm` $\rightarrow$ `norm_position="post"`, `--no-rope` $\rightarrow$ `use_rope=False`, `--ffn-silu` $\rightarrow$ `ffn_type="silu"` (pass `--d-ff 4*d_model` yourself, as the assignment asks). Non-default knob values are only forwarded if the constructor accepts them, otherwise the script exits with a message naming the missing parameter (`-tie-embeddings` is reserved this way; `model.py` does not implement it).
- **Optimizer / schedule:** `cs336_basics.optimizer.AdamW` (`--lr --beta1 --beta2 --eps --weight-decay`); learning rate set every step from `get_lr_cosine_schedule(step, lr, min_lr, warmup_iters, cosine_iters)` with `min_lr` defaulting to `lr/10` and `cosine_iters` defaulting to `--max-steps`, so decay ends exactly at the last step; global-norm gradient clipping (`--grad-clip`, default 1.0) via `nn_utils.gradient_clipping`.
- **Budget:** `--max-steps N` or `--total-tokens T` (`steps = T / (B*m)`, the form the assignment uses: 327,680,000 tokens); `--max-wallclock-min` stops (and checkpoints) after a time limit — the leaderboard's 45-min cap.
- **Data:** memmapped as above; a sanity check aborts if the first 1M tokens contain an `id` $\geq$ `--vocab-size` (catches wrong dtype / wrong vocab).
- **Evaluation:** every `--eval-interval` steps (and at the end) the mean cross-entropy over `--eval-batches` validation batches drawn with a fixed seed (`--eval-seed`), so every evaluation and every run sees the same validation tokens and curves are comparable. Eval time is excluded from the throughput numbers but included in `elapsed_s`.
- **Logging:** every `--log-interval` steps one JSON line is appended to `RUN/log.jsonl` and echoed to the console. Fields: `step`, `train_loss`, `lr`, `elapsed_s` (wall-clock incl. previous segments on resume), `tokens_seen`, `tokens_per_s`, `step_ms`, `grad_norm` (pre-clip global norm), plus `val_loss`, `val_perplexity` on eval steps. `RUN/config.json` records all arguments and the parameter count. `--wandb` mirrors the same records to Weights & Biases (`--wandb-project`); the import happens only when the flag is given, so no network dependency by default.

- **Checkpoints:** `RUN/ckpt_step<N>.pt` every `--ckpt-interval` steps and `RUN/ckpt_final.pt` at the end (also on Ctrl-C or wall-clock stop); `--resume PATH` restores model, optimizer, step, elapsed time and tokens seen.
- **Speed knobs:** `--compile` (torch.compile; verified on CPU), `--dtype bf16` (autocast on CUDA; loss is computed in fp32 from up-cast logits), `--tf32`.
- **Debugging:** `--overfit-one-batch` reuses the first batch; with the 1.4M-param smoke model the loss goes $9.2 \rightarrow 0.006$ in 200 steps, the standard sanity check that the model/loss/optimizer path is wired correctly.

Smoke test (CPU, d_model 64, 2 layers, ctx 64, B 16, TinyStories-valid subset tokenized with the 10k vocab): loss $9.03 \rightarrow 7.03$ over 60 steps; resuming from `ckpt_step40.pt` continued at step 40 with elapsed 96 s and reached step 70; `generate.py` produced text from the resulting checkpoint. Practical note for the shared C12 box: with 4 OMP threads on cores already busy with other jobs each tiny op cost ~9 ms of thread synchronisation (fwd 667 ms vs 70 ms single-threaded), so use `OMP_NUM_THREADS=1` for CPU smoke tests there.

6 Decoding (decoding.py, scripts/generate.py)

`generate(model, prompt_ids, max_new_tokens, temperature, top_p, eos_token_id, device, context_length, generator)` loops: feed the last `context_length` tokens, take the last row of logits `v`, sample, append; stop at `eos_token_id` (`<|endoftext|>`) or after `max_new_tokens`; returns the new ids only.

- Temperature (eq. 23): $\text{softmax}(v/\tau)$ computed as $\exp((v/\tau) - \max) / \Sigma$; `temperature == 0` takes the argmax (the $\tau \rightarrow 0$ limit).
- Top-p (eq. 24): sort `q` descending, cumulative sum `c`; drop token `j` iff `c_j - q_j ≥ p` (the mass *before* it already reaches `p`), which keeps the smallest prefix whose mass is $\geq p$ including the token that crosses the threshold; the top-1 token is always kept; scatter back and renormalise. Checked: `q = (.5, .3, .15, .05)`: $p=0.5 \rightarrow (1, 0, 0, 0)$; $p=0.8 \rightarrow (.625, .375, 0, 0)$; $p=0.81 \rightarrow (.526, .316, .158, 0)$; sampling 1000 draws with $p=0.8$ only ever returned ids `{0, 1}`.
- Sampling is `torch.multinomial` with an optional `torch.Generator` for reproducibility.
`generate_text(model, tokenizer, prompt, ...)` is the string wrapper.

`scripts/generate.py --checkpoint CKPT --vocab vocab.json --merges merges.txt --prompt "..." --max-new-tokens 256 --temperature T --top-p P [--seed S] [--num-samples K] [--device]` rebuilds the model from the config stored in the checkpoint (`extra.model_config`; overridable per flag), loads weights, and prints prompt + continuation, with token count / stop reason on stderr.

§7 generate deliverable

Filled in `writeup/section7_experiments.md`, section *generate*: a 300-token dump and a 170-token story from the best TinyStories checkpoint (`ts_full_lr2e-3`, full-validation loss 1.391; `scripts/generate.py ... --temperature 0.8 --top-p 0.9`), plus the discussion of the factors that change output quality (temperature / top-p, training tokens / validation loss, dataset, prompt). The same section also lists the OpenWebText samples (`main_experiment`).

Section 7 - Experiments

All runs: `writeup/experiment_log.md` (one row per run, config / tokens / steps / wall-clock / final losses);
curves: `writeup/figures/*.png` (every figure exists in a `_steps` and a `_wallclock` version). Code:
`cs336_basics/train.py` (training loop + jsonl logging, §5), `scripts/plot_runs.py` (curves + summary table from the jsonl logs), `scripts/eval_checkpoint.py` (loss over the *entire* validation set),
`scripts/sec7_make_figures.sh` (the exact plot commands for every figure below), `scripts/generate.py` (§6 decoding). Run directories with logs / configs / checkpoints are kept outside the repository
(`/home/c12/workspace/claude_workspace/cs336_a1/sec7-exp/runs/`).

Setup and compute accounting

- Hardware: one RTX 5080 (16 GB, shared; our jobs were capped at ≈ 6 GB and one training process at a time), 4 CPU cores, `bf16` autocast + `TF32` + `torch.compile` (Inductor). Not a B200, so the assignment's "B200-hours" do not apply; we report wall-clock on this card everywhere. Baseline throughput for the 17M-parameter model (22.7M with embeddings): **$\approx 330\text{--}350\text{k tokens/s at }B=32$** (`bf16+compile`; 140k without compile, 95k in `fp32`), i.e. the official 327.68M-token budget takes ≈ 17 min, the low-resource 40.96M-token budget ≈ 2.5 min.
- Baseline configuration (§7.2.1): vocab 10,000 (`out/ts10k`), context 256, `d_model` 512, `d_ff` 1344, 4 layers, 16 heads, RoPE $\Theta=10^4$, AdamW $\beta=(0.9, 0.95)$, $\epsilon=1e-8$, weight decay 0.1, gradient clipping at global norm 1.0, linear warmup for 5% of the steps then cosine decay to $\alpha_{\min} = \alpha_{\max}/10$ ending exactly at the last step, batch 32.
- Two token budgets: **full** = 327,680,000 tokens (40,000 steps at $B=32$) for the final TinyStories model, and **short** = 40,960,000 tokens (5,000 steps at $B=32$, the assignment's low-resource budget) for every sweep/ablation. Time was not the binding constraint (the whole section used ≈ 4 GPU-hours), the short budget was chosen so that sweeps stay comparable with the assignment's reference numbers (1.80 val loss at 5,000 steps).
- Validation loss during training is measured every 250 steps (short) / 1,000 steps (full) on the same 40 fixed random validation batches (327,680 tokens, seed 1234) for every run, so all curves are on identical data. Final numbers for the headline models are recomputed on the **entire** validation set (5,465,856 tokens in non-overlapping 256-token windows) with `scripts/eval_checkpoint.py`; the fixed-batch estimate is ≈ 0.01 optimistic (e.g. 1.6485 vs 1.6597 for the `lr=3e-3` short run).
- Data: `data/ts_train_10k.npy` (541,229,347 tokens), `data/ts_valid_10k.npy` (5,465,883), `data/owt_train_32k.npy` (2.73B tokens, vocab `out/owt32k`), `data/owt_valid_32k.npy` (66.4M) — `uint16` memmaps produced by `scripts/tokenizer_experiments.py encode` (§2.7).

A bug found on the way (`torch.compile` + `bf16` on the RTX 5080)

The very first compiled `bf16` run produced NaN from step 1, while eager `bf16`, compiled `fp32` and eager `fp32` were all fine. Bisection (`dbg_nan*.py`, kept in the run workspace) showed the forward is finite but Inductor's fused *backward* of `einsum(Q,K)/sqrt(d_k)  $\rightarrow$  softmax` in `bf16` returns non-finite gradients for `Q` and `K`, with or without the causal mask. Computing the attention softmax in `fp32` (`scores.float(), probs.to(V.dtype)`) — which is what autocast does for `softmax` anyway — fixes it and costs nothing measurable. The fix is now in `model.py` (`scaled_dot_product_attention`); the runs below were launched through a thin wrapper that applied the identical patch before it was merged, so all of them use exactly the current model code.

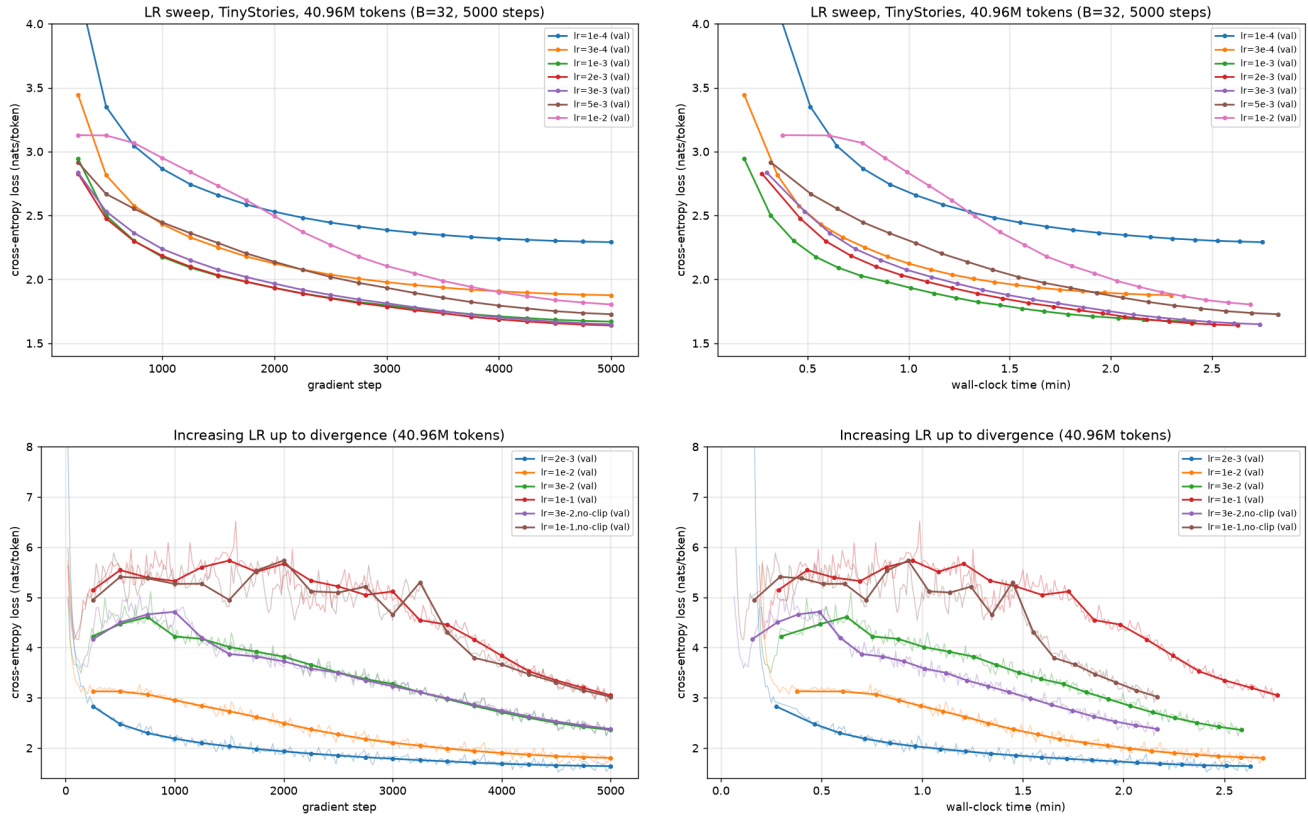
experiment_log

Deliverable: `writeup/experiment_log.md` + `writeup/figures/`. The infrastructure is the jsonl logger in `cs336_basics/train.py` (every 20 steps: `step`, `train_loss`, `lr`, `elapsed_s`, `tokens_seen`, `tokens_per_s`, `step_ms`, `grad_norm`; every eval: `val_loss`, `val_perplexity`; `elapsed_s` survives `--resume`, so wall-clock curves are continuous), `config.json` per run (all arguments + parameter count), and `scripts/plot_runs.py`, which draws any set of runs against gradient steps *and* wall-clock time and prints the markdown summary table used in the log (matplotlib is not a project dependency; run it with any interpreter that has it). Each run's raw logs are kept under `runs/<group>/<name>/{log.jsonl,config.json,stdout.log,ckpt_final.pt}` (workspace, not in git;

checkpoints are 270 MB each). `--wandb` mirrors the same records to W&B but was not used here (no network dependency).

learning_rate

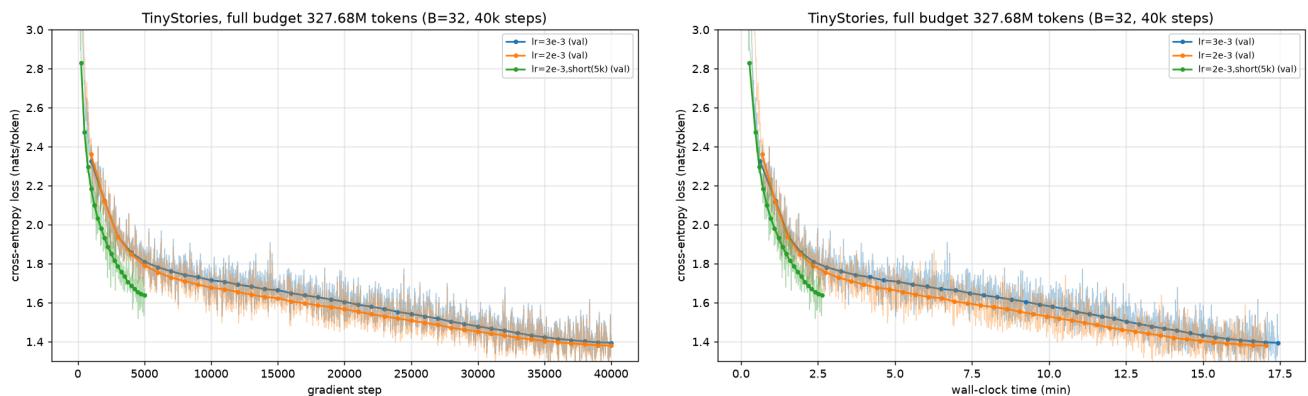
Search strategy. Coarse-to-fine grid at the short budget (5,000 steps, B=32), warmup 250, cosine to $\alpha/10$: first a $\times 3$ grid $1e-4 \dots 1e-2$, then the neighbours of the minimum (2e-3, 5e-3), then $\times 3$ steps upward until divergence (3e-2, 1e-1). One seed each; the differences between adjacent LRs near the optimum (0.01 nats) are at the noise floor, so "best" means "best-or-tied". Figures: `figures/lr_sweep_{steps,wallclock}.png`, `figures/lr_divergence_{steps,wallclock}.png`.



peak LR	1e-4	3e-4	1e-3	2e-3	3e-3	5e-3	1e-2	3e-2	1e-1
val loss @5,000 steps (40.96M tokens)	2.291	1.876	1.669	1.639	1.648	1.727	1.803	2.359	3.049

The optimum is broad (2e-3 – 3e-3, $\times 10$ above the "safe" 3e-4 many people start with); at 1e-3 the model is still clearly under-trained at 5,000 steps, at 5e-3 and above the early phase gets worse (higher loss during/after warmup, more noise) and the run never catches up within the budget.

(a) Full-budget model. `ts_full_lr3e-3` (and `ts_full_lr2e-3`): baseline architecture, 327,680,000 tokens = 40,000 steps $\times$ 32 $\times$ 256, warmup 2,000, cosine to $\alpha/10$ at step 40,000. Figure `figures/ts_full_{steps,wallclock}.png`.



run	LR (peak → min)	wall-clock	tokens/s	val loss, 40 fixed batches (last / best)	val loss, full validation set
ts_full_lr3e-3	3e-3 → 3e-4	17.4 min	≈305-340k	1.3945 / 1.3945	1.4066 (ppl 4.08)
ts_full_lr2e-3	2e-3 → 2e-4	17.1 min	≈305-345k	1.3796 / 1.3796	1.3909 (ppl 4.02)

Both meet the ≤ 1.45 target; `ts_full_lr2e-3/ckpt_final.pt` (1.391) is the model used for generate below. The two full runs confirm the sweep's ranking ($2e-3 \geq 3e-3$), and a second seed at the short budget agrees within noise ($2e-3$: 1.639 / 1.628; $3e-3$: 1.648 / 1.634 for seeds 0 / 1). On the wall-clock axis the full run reaches the short run's final loss (1.64) after ≈ 4 min and keeps improving almost linearly in log-time to 1.39 at 17 min; the schedule (cosine to $\alpha/10$ ending at the last step) gives the characteristic final drop of ≈ 0.05 in the last 15% of the run.

(b) Edge of stability. With gradient clipping (norm 1.0) *no* learning rate produced NaN: $3e-2$ and $1e-1$ "diverge" in the sense that the loss, after falling to ≈ 3.5 – 4.2 in the first 100 steps, turns around and climbs to 4.6 ($3e-2$) / 5.7 ($1e-1$) as the warmup pushes the LR past $\approx 1.5e-2$, stays there for thousands of steps, and only recovers partially once cosine decay brings the LR back below $\approx 1e-2$ (final 2.36 / 3.05). In both runs the turning point is at $LR \approx 1.2e-2$ – $3e-2$ (step ≈ 100 of the 250-step warmup for $3e-2$, step ≈ 80 for $1e-1$), which is also where the un-clipped runs blow up: without gradient clipping (`--grad-clip 0`) the picture is the same — $3e-2$ ends at 2.379 (vs 2.359 clipped) and $1e-1$ at 3.020 (vs 3.049) — because AdamW's per-coordinate normalisation already bounds the update size, so "divergence" for this optimizer does not look like NaN but like a loss that shoots up towards the unigram entropy (≈ 4.5 – 5.7 nats) and stays there while the LR is high. (The one configuration that did produce NaN is the no-RMSNorm model at $3e-3$, see the ablation below.) So the instability threshold of this model/optimizer is $\approx 1e-2$ – $3e-2$, and the best learning rate ($2e-3$ – $3e-3$) sits ≈ 5 – $10\times$ below it — consistent with the folk wisdom that one should train close to, but not at, the edge of stability: $1e-2$ ($\approx$ threshold/2) still trains but is *worse* than $3e-3$ at every step of the curve, and $3e-4$ (threshold/50) is far too slow. Convergence speed grows with LR up to $\approx$ threshold/5 and then degrades because the noise floor (loss oscillation $\propto$ LR) and warmup damage dominate before the higher LR can pay off.

generate

Best checkpoint: `ts_full_lr2e-3/ckpt_final.pt` (full-validation loss 1.391). Command: `scripts/generate.py --checkpoint ... --vocab out/ts10k/vocab.json --merges out/ts10k/merges.txt --prompt "Once upon a time" --max-new-tokens 300 --temperature 0.8 --top-p 0.9 --seed 7 --no-eos-stop` (without `--no-eos-stop` every sample ends at `<|endoftext|>` after 140–200 tokens, because TinyStories stories are short; the 300-token dump below therefore contains the end of one story and the start of the next).

```

Once upon a time, there was a little boy named Tim. Tim loved to play outside. One day, he found a small
rock. He was very happy. He thought it was a magic rock.
Tim showed the rock to his friend, Sam. Sam said, "Wow! That's a cool rock. Can I see it?" Tim and Sam
said, "Yes, but be careful. The rock can be dangerous."
Tim and Sam played with the rock. They took turns to push it with their hands. They had so much fun. At
the end of the day, Tim and Sam were tired. They said goodbye to the rock and went home. Tim was glad he
had a fun day with his friend.
<|endoftext|>
One day, a little boy named Tim went to the park with his mom. Tim saw a big, orange ball near a tree. He
wanted to play with the ball, but it was too high. Tim was sad.
A nice girl named Sue saw Tim and asked, "Why are you sad?" Tim said, "I want to play with the orange
ball, but it is too high." Sue thought for a moment and said, "I have an idea! Let's play together."
Sue and Tim played with the orange ball. They threw it back and forth. They had lots of fun. But then,
something unexpected happened. The ball started to grow bigger and bigger!
"Wow, it's so big now!" said Tim. "Let

```

Same checkpoint, `--temperature 0.8 --top-p 0.9 --seed 1` (stopped at `<|endoftext|>`, 170 tokens):

Once upon a time, in a small house, there lived a little girl named Mia. Mia loved to play with her toys and run around the house. One day, she found a magic word in the house. The word was " math." She did not know what it meant, but she wanted to learn. Mia decided to learn math from the magic word. She went to her mom and showed her the magic word. Her mom said, "This is math. It helps us learn." Mia was very happy to learn this new word. She wanted to learn more. Mia's mom showed her the magic word. She said, "Bally, now you can learn math too!" Mia was so excited. She loved learning new things and learning with her mom. From that day on, Mia always tried to learn math and it was always fun.

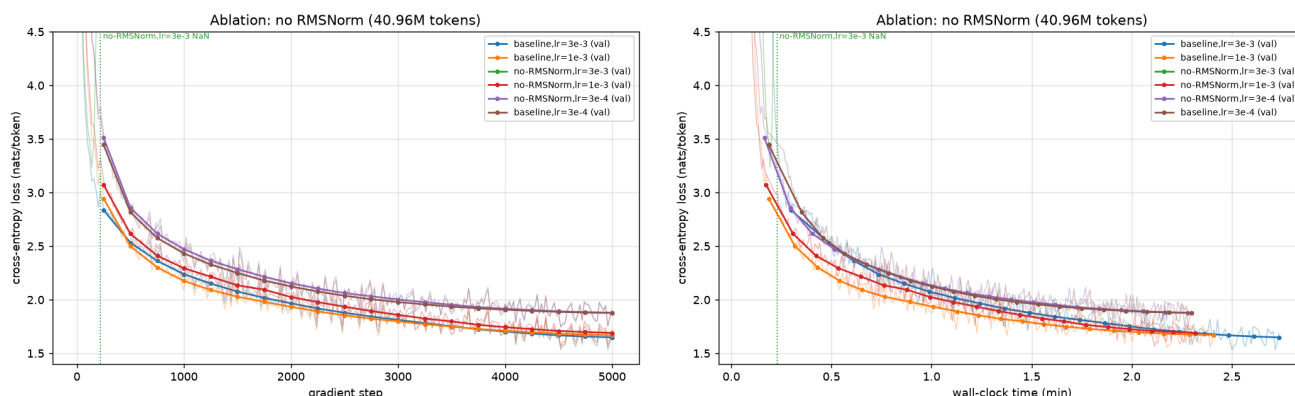
Fluency. Grammatical, on-topic children's-story English with consistent character names, dialogue punctuation and a moral-style ending, i.e. at least as good as the assignment's reference sample; the residual errors are semantic (a "magic word ... math", "Mr. Bear was very rough and could not talk" followed by Mr. Bear talking) rather than syntactic — the 17M model has learned the surface form and the local plot template of TinyStories but not reliable world/state tracking over a whole story.

Factors that change the quality (all observed with this checkpoint / its siblings):

1. *Temperature / top-p.* $T=1.0$, $\text{top-p}=1.0$ (pure sampling) produces stories that drift into nonsense ("there was a cone. This cone was called candy ... the cone still could not decorate her") because low-probability tokens are drawn; $T=0.3$ or greedy ($T=0$) is perfectly fluent but collapses onto the same template every time ("there was a little girl/boy named Lily/Tim ... loved to play with ... One day ..."), and greedy tends to repeat phrases. $T \approx 0.7-0.9$ with $\text{top-p} \approx 0.9$ is the sweet spot: nucleus sampling removes the tail that causes the semantic derailments while keeping variety.
2. *Training tokens / validation loss.* The 40.96M-token checkpoint of the same architecture ($\text{lr_sweep}/\text{lr}2e-3$, val 1.64) with the same decoder settings and seed yields the same kind of story but with more non-sequiturs per sentence ("The mouse said, 'Please open the box, and you will open it.'"); the $3e-4$ short run (val 1.88) is worse again ("The mouse was happy and said, 'Hello, I am Tim.'"). Loss differences of 0.2–0.5 nats are clearly visible in the samples.
3. *Model size / dataset* (see `main_experiment` below): the identical model trained on OpenWebText with the same budget reaches loss ≈ 4.16 and its samples are far less coherent — with a much broader domain, 17M parameters and 327M tokens are simply not enough.
4. *Prompt.* The model has only ever seen stories, and "Once upon a time" is the canonical opening; unusual prompts push it off-distribution quickly.

layer_norm_ablation

`--no-rmsnorm` removes the two block norms and the final norm (`norm="none"`). Figure `figures/ablation_rmsnorm_{steps,wallclock}.png`.

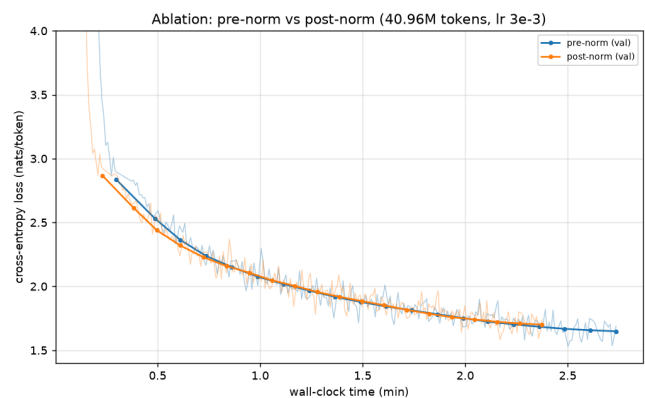
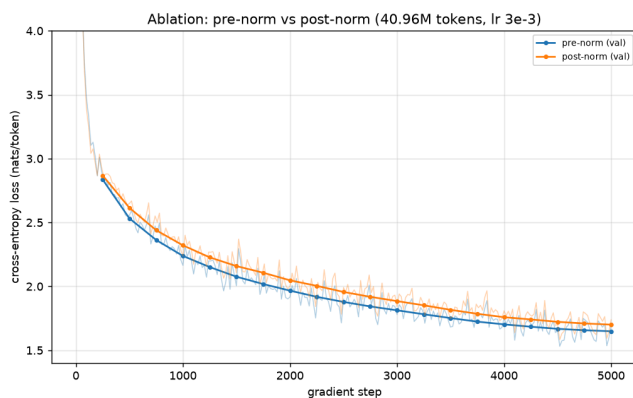


run	LR	outcome	val loss @5k
baseline	3e-3	fine	1.648
no RMSNorm	3e-3 (previous optimum)	diverges : loss falls to 3.2 by step 140, grad-norm spikes to 32 at step 160 (LR $\approx 1.9\text{e-}3$ during warmup), loss $4 \cdot 10^4$ at step 200, NaN from step 220	NaN
no RMSNorm	1e-3	trains; noisier grads (final grad-norm 0.9 vs 0.3–0.4)	1.690
no RMSNorm	3e-4	trains, but slow	1.879
baseline	1e-3 / 3e-4		1.669 / 1.876

Comment: without normalisation the residual stream's scale is unconstrained, so the effective step size of Adam's per-parameter normalised update grows with the activations, and the model becomes unstable at $\approx 1/2$ the LR that the pre-norm model tolerates comfortably. A $3\times$ lower LR restores stability, and then the loss is only 0.02–0.04 worse than the baseline at the same LR — i.e. at this depth (4 layers) RMSNorm buys mostly *stability / a wider usable LR range*, not a better optimum, and its cost is a smaller maximum learning rate for the un-normalised model, which is exactly what makes it worse at a fixed step budget (1.690 vs the baseline's 1.639–1.648 at its own best LR).

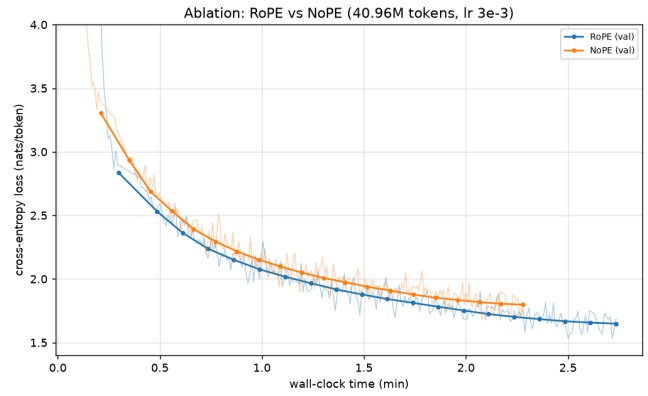
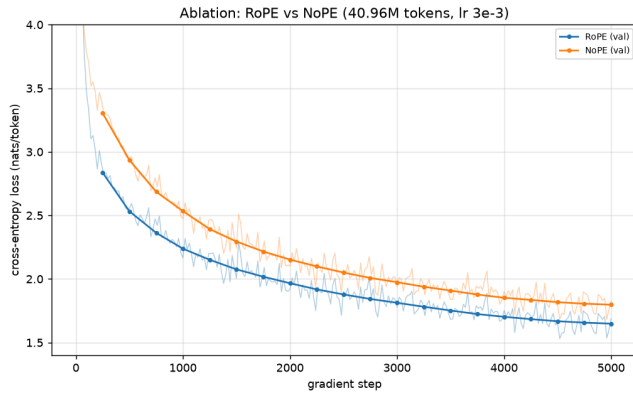
pre_norm_ablation

--post-norm: $z = \text{RMSNorm}(x + \text{Attn}(x))$, $y = \text{RMSNorm}(z + \text{FFN}(z))$, no final norm. Figure figures/ablation_postnorm_{steps,wallclock}.png. At the same LR (3e-3) the post-norm model trains stably (no NaN, no spikes) but is worse from the first evaluation on and ends at **1.701 vs 1.648** (pre-norm), i.e. the gap does not close with training. Post-norm re-normalises the residual stream after every block, so the identity path from the embedding to the output is broken (each block's output is scaled by the norm), gradients to early layers are attenuated and the model behaves more like a deep non-residual network — slower optimisation and a lower tolerance to large learning rates (with only 4 layers we see the slower optimisation but not the instability; the classic result is that post-norm needs warmup and small LR's at depth). Pre-norm keeps a clean residual path, which is why it is the consensus choice.



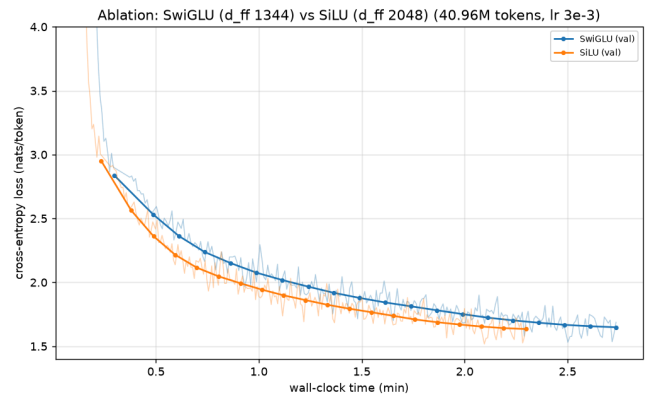
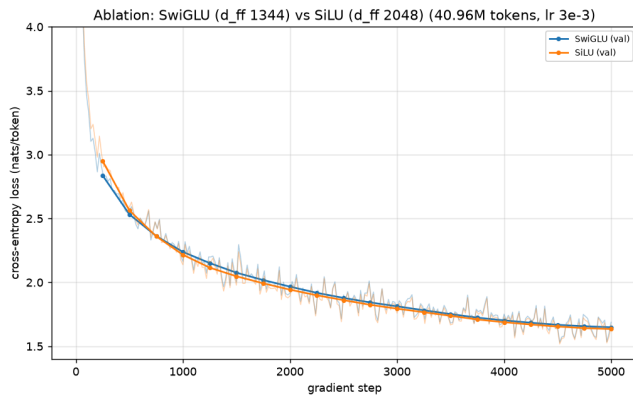
no_pos_emb

--no-rope (NoPE: no positional information at all; the causal mask is the only position signal). Figure figures/ablation_nope_{steps,wallclock}.png. NoPE trains stably but is worse throughout: **1.798 vs 1.648** at 5k steps ($\Delta = 0.15$ nats, i.e. more than the gap between the best LR and a $3\times$ too small LR). A causal decoder can in principle recover position (e.g. by counting via attention to a BOS-like token), but it has to *learn* that representation, spending capacity and steps on it, and the learned signal is less precise than RoPE's explicit relative rotation — a large handicap for a task where local order (subject-verb-object, quotes) matters and the model is small and trained briefly. With a longer budget the gap would shrink but not vanish (RoPE's inductive bias is simply the right one for language).



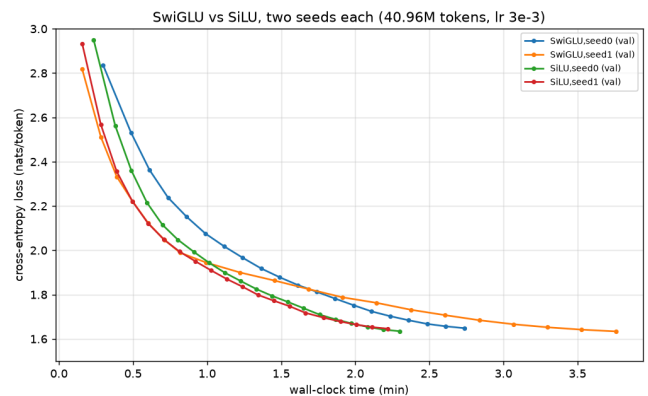
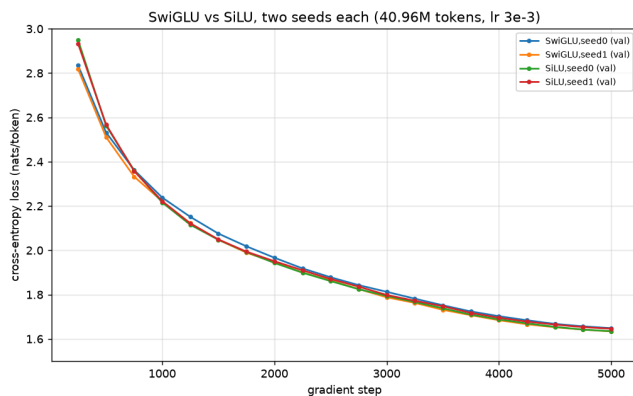
swiglu_ablation

--ffn silu --d_ff 2048 (FFN(x) = $W_2 \text{SiLU}(W_1x)$ with $d_{\text{ff}} = 4 \cdot d_{\text{model}}$, 2 matrices) vs the SwiGLU baseline ($d_{\text{ff}} 1344$, 3 matrices). Parameter counts: 22.83M vs 22.70M (FFN 2.10M vs 2.06M per layer) — matched within 0.6%. Figure `figures/ablation_swiglu_{steps,wallclock}.png`.



FFN	params	seed 0	seed 1	mean
SwiGLU, $d_{\text{ff}} 1344$ (baseline)	22.70M	1.648	1.634	1.641
SiLU (no gate), $d_{\text{ff}} 2048$	22.83M	1.635	1.646	1.640

(second seeds: `figures/ablation_swiglu_seeds_{steps,wallclock}.png`)

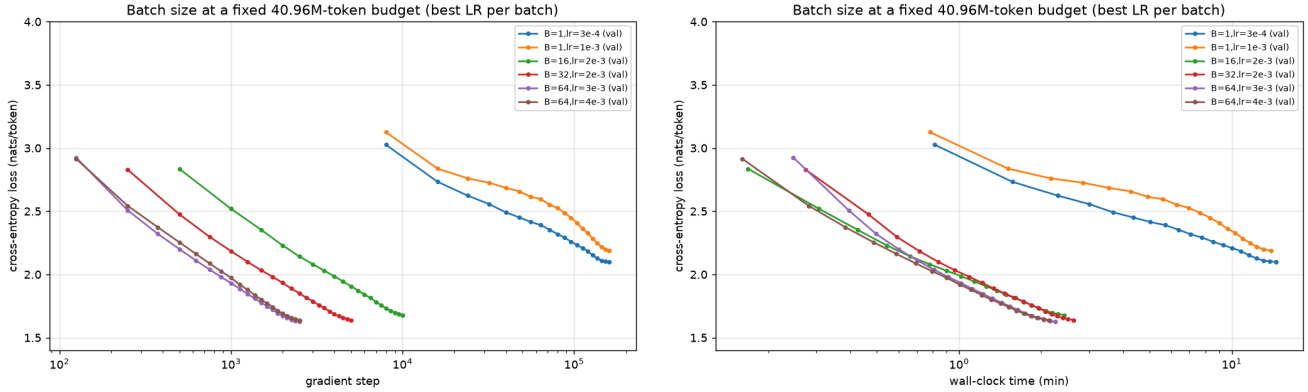


At this scale and budget the two are **indistinguishable**: the seed-to-seed spread (0.014) is as large as the difference between the architectures, and the curves lie on top of each other for the whole run. Throughput is also the same (both $\approx 330\text{--}340\text{k tok/s}$: the gated FFN has 3 smaller matmuls, the plain one 2 larger ones, same FLOPs). So we do not reproduce Shazeer's "GLU variants are better" here — his gains ($\approx 0.02\text{--}0.05$ in log-perplexity for T5-base) were measured on much larger models and far longer training; with a 4-layer 17M-parameter model trained for 40M tokens on TinyStories the FFN is not the bottleneck (the loss is dominated by what the model has seen, not by FFN expressivity), and the extra element-wise gate neither helps nor hurts optimisation. What the ablation *does* confirm is that SwiGLU's parameter re-allocation

(three matrices with $d_{\text{ff}} = 8/3 \cdot d_{\text{model}}$ instead of two with $4 \cdot d_{\text{model}}$) is "free": you get the gating mechanism, which pays off at scale, at equal parameter count, FLOPs and throughput.

batch_size_experiment

Fixed budget of 40.96M tokens for every batch size (steps = $40.96\text{M} / (B \cdot 256)$), warmup 5% of the steps, cosine to $\alpha/10$, LR re-tuned per batch size with two values each (the B=32 optimum $3\text{e-}3$ and a $\sqrt{B}$ -scaled guess), validation on the same 327,680 tokens (--eval-batches 1280/B). Figure figures/batch_size_{steps,wallclock}.png (step axis is log-scaled because the step counts span 2,500 ... 160,000).



batch	steps	LR tried → best	val loss @40.96M tokens	wall-clock	tokens/s	peak VRAM
1	160,000	1e-3 → 2.188, 3e-4 → 2.099	2.099	14.5 min	≈47-52k	0.47 GiB
16	10,000	3e-3 → 1.695, 2e-3 → 1.679	1.679	2.4 min	≈305k	1.56 GiB
32	5,000	3e-3 → 1.648, 2e-3 → 1.639	1.639	2.6 min	≈290-335k	2.72 GiB
64	2,500	4e-3 → 1.639, 3e-3 → 1.625	1.625	2.3 min	≈350k	5.05 GiB
128 / 256	—	—	OOM under our ≈6 GB share (B=128 needs ≈10 GB)	—	—	—

(The B=32 runs happened to overlap with a CPU-heavy tokenisation job on the shared box, hence their lower tokens/s; other B=32 runs reached 325-345k.)

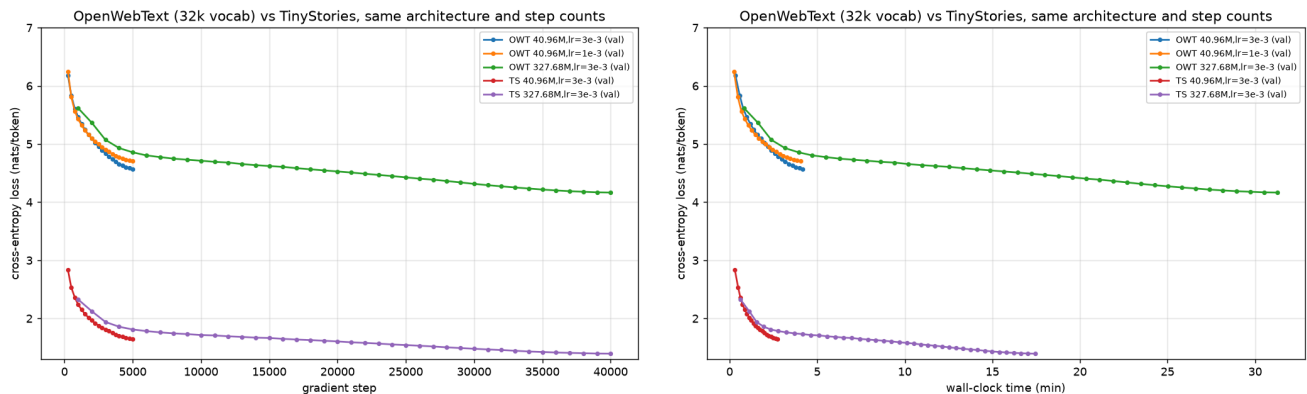
Memory limit. Peak allocated memory (bf16 autocast, compiled, AdamW): B=1 0.47 GiB, 16 1.56 GiB, 32 2.72 GiB, 64 5.05 GiB (≈6.5 GB as seen by nvidia-smi including the CUDA context / compile workspace); B=128 ran out of memory at ≈8.5 GiB allocated (needs ≈10 GB, dominated by the fp32 logits and their softmax/CE intermediates: $128 \cdot 256 \cdot 10,000 \cdot 4 \text{ B} = 1.3 \text{ GB}$ per copy) — above the ≈6 GB share we had on the 16 GB card, so B=64 was the largest batch we could run and 128/256 were skipped.

Findings. (i) *Throughput*: B=1 is 6-7× slower per token than B≥16 (≈50k vs 300-350k tok/s) because every kernel is launch/latency-bound and the matmuls are tiny; from B=16 upward the GPU is already saturated by this small model (305k → 335k → 350k tok/s for 16 → 32 → 64), so larger batches buy only ~10% more throughput. (ii) *Sample efficiency at a fixed token budget*: the loss after 40.96M tokens improves monotonically with batch size, but with strongly diminishing returns: 2.10 (B=1) >> 1.68 (16) > 1.64 (32) > 1.625 (64). B=1 is dramatically worse even with a re-tuned (much smaller) LR: with Adam's per-coordinate normalisation the gradient noise at B=1 is so large that a small LR is needed, and 160k tiny steps then cover the same tokens with far less progress per token — the noise, not the number of updates, is what limits it. Between 16 and 64 we are below the *critical batch size* for this model/dataset (the regime where doubling B halves the number of steps at almost no loss penalty), so larger batches are pure win: same tokens, fewer and cheaper steps, slightly better loss. (iii) *Learning rate*: the best LR grows with batch size ($3\text{e-}4 \rightarrow 2\text{e-}3 \rightarrow 2\text{e-}3 \rightarrow 3\text{e-}3$), roughly like $\sqrt{B}$ as expected for Adam, so the LR must be re-tuned when the batch changes. (iv) *So do we always want large batches?* Not unconditionally: past the critical batch size more sequences per step would stop reducing the number of steps needed (the loss-vs-tokens curve

would get worse), and at this model size we already gain little throughput beyond $B=32$; the practical limit here was memory (fp32 logits of $B \times 256 \times 10,000$), and for a fixed *wall-clock* budget $B=32-64$ is the sweet spot on this GPU. Larger batches also mean fewer optimizer steps for the same data, which is exactly the regime where the LR schedule / warmup starts to matter more.

main_experiment (OpenWebText)

Same architecture (vocab 32,000, out/owt32k; everything else identical), same step counts as the TinyStories runs: 5,000 steps $\times$ B32 (40.96M tokens) at two learning rates, then the full 40,000 steps (327.68M tokens) at the better one. Figure figures/owt_{steps,wallclock}.png (TinyStories runs of the same budget overlaid).



run	data	LR	steps $\times$ B	tokens	wall-clock	tokens/s	val loss (fixed batches)	full-val-set loss
owt40M_lr3e-3	OWT	3e-3	5,000 $\times$ 32	40.96M	4.2 min	181k	4.572	—
owt40M_lr1e-3	OWT	1e-3	5,000 $\times$ 32	40.96M	4.1 min	181k	4.708	—
owt_full_lr3e-3	OWT	3e-3	40,000 $\times$ 32	327.68M	31.3 min	178k	4.165	4.157 (ppl 63.9; first 25.6M val tokens)
lr3e-3 (TS)	TinyStories	3e-3	5,000 $\times$ 32	40.96M	2.7 min	≈ 330 k	1.648	1.660
ts_full_lr3e-3 (TS)	TinyStories	3e-3	40,000 $\times$ 32	327.68M	17.4 min	≈ 330 k	1.395	1.407

The OWT model has 45.2M parameters (the 32k vocabulary adds 2×16.4 M embedding / LM-head weights to the same 17M-parameter body) and runs at ≈ 178 k tok/s instead of ≈ 330 k: the vocabulary projection and the fp32 cross-entropy over 32,000 logits dominate the step time of a model this small. The same LR (3e-3) was best on both datasets.

Interpreting the losses. The OWT loss is 4.16 vs 1.39 on TinyStories after the identical number of steps and tokens, i.e. perplexity 64 vs 4. Three things are mixed into that number. (1) *Different tokenizers = different units.* A 32k-vocab token carries more text than a 10k-vocab token (4.37 vs 4.12 bytes/token on these corpora), so per-token losses are not directly comparable; per byte the numbers are 0.95 vs 0.34 nats/byte — still $2.8\times$ apart, so most of the gap is real, not a unit effect. (2) *The data are intrinsically harder.* TinyStories is synthetic (GPT-3.5/4-generated) text with a ~ 1.5 k-word vocabulary, a handful of plot templates and near-perfect regularity, so a tiny model can approach its entropy; OWT is open-domain web text — names, numbers, dates, URLs, code, dozens of registers — whose irreducible entropy under any small model is far higher. A loss of 4.16 nats/token (≈ 0.95 nats/byte ≈ 1.37 bits/byte) is in fact a normal number for a ~ 50 M-parameter model at this budget; GPT-2-small-class models reach $\approx 3.0-3.3$ nats/token on OWT with $100\times$ more compute. (3) *The model is severely under-trained for OWT.* 327M tokens is 0.6 epochs of TinyStories but 0.12 epochs of our 2.7B-token OWT sample, and the OWT curve is still descending steeply at the end ($4.31 \rightarrow 4.16$ in the last 25% of steps, with the fixed-batch loss dropping ≈ 0.15 per 10k steps) whereas the TinyStories curve is flattening. So the loss should be read as "how much of the web can 17M non-embedding parameters absorb in 30 minutes" — much less than "how much of TinyStories" — and it is only meaningful relative to other models on the *same* tokenizer and validation set (which is exactly why the leaderboard fixes both).

Generated text (owt_full checkpoint, prompt "The", T=0.8, top-p 0.9, 256 tokens):

The Oklahoma City Commission is conducting a hearing on the case.

The document was obtained by phone from the AirBurna County Sheriff's Office. The internal review document was obtained by phone from the Star Tribune.

The document was released on October 13, 2015.

The document, which was released on October 16, said it was unable to provide details on what went into the package because the document was released and the document was originally leaked.

The document was compiled by a federal criminal complaint with the state Department of Education and Education.

The document includes the information obtained by the company's internal review.

The document states that the document was made using the documents with the purpose of the document and that it was defamatory.

The document reads, "Alberta said the document was a master and was personally found to have used a bomb in the 30s. He said the document was public and that it was never in the interest of the public."

The document states that the document was removed from the document, but that it was still not on release.

The document was sent by a state agency that reportedly says the document was collected and that it was intended to send out information on the document.

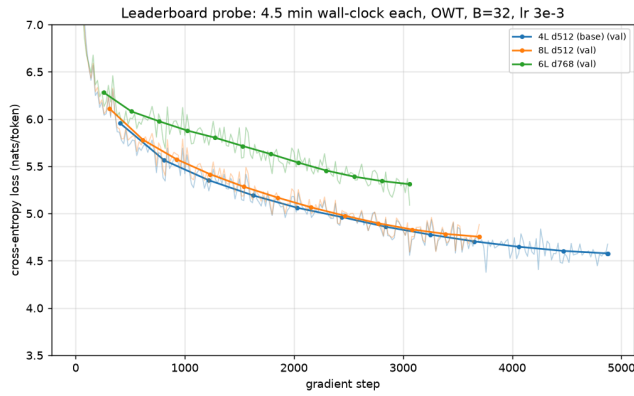
(A second sample with the TinyStories prompt "Once upon a time, ..." turned into a hospital anecdote: "the resident had been told that the first two patients would be treated with each other ..."; pure sampling T=1, top-p=1 degrades into word salad — "He mode said he needs to 'blame the stock action'".)

Fluency. Locally the text is English: sentences are grammatical, the register (wire-service news) is consistent, and there is even a plausible document structure (dateline-like first sentence, quotes, "[3]"-style footnote markers under pure sampling). Globally it is nonsense: it loops on a few high-frequency frames ("The document was ..." ×10), entities and facts change every sentence, and there is no discourse coherence beyond ~15 tokens. It is clearly worse than the TinyStories output even though model, tokens and compute are identical, because (i) OWT's distribution is enormously wider — the model has to spread the same 17M non-embedding parameters over news, forums, fiction, code and reference text and never sees enough of any register to model it well (0.12 epochs, most bigrams appear once), whereas TinyStories concentrates 327M tokens on a ~1.5k-word, few-template domain that a 4-layer model can essentially memorise; (ii) the useful context is much longer on OWT — a 256-token window rarely contains the information needed to continue a news article coherently, while a whole TinyStory fits in it; (iii) the loss gap itself: at 4.16 nats/token the model's next-token distribution is still very flat (perplexity 64), so nucleus sampling has to draw from many mediocre candidates at every step and errors compound quickly, whereas at 1.39 (perplexity 4) most tokens are near-deterministic and a story stays on track. Also, OWT contains lots of boilerplate ("The document was released on ..."), which a weak model over-produces because it is the cheapest way to lower the loss.

leaderboard

Caveat first. The rules say 45 minutes *on a B200*; we have one RTX 5080 shared with other jobs (≈6 GB VRAM, one process), whose bf16 throughput on this model is roughly 5–8× below a B200's, so our result is a "45 minutes of 5080 wall-clock" best effort and is *not comparable* to the class leaderboard numbers (which see several times more tokens in the same time). What we can honestly report is the loss reached in 45 min on this card and the reasoning behind the configuration.

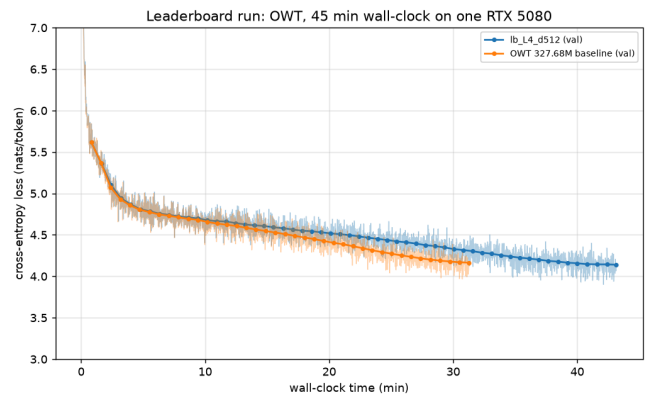
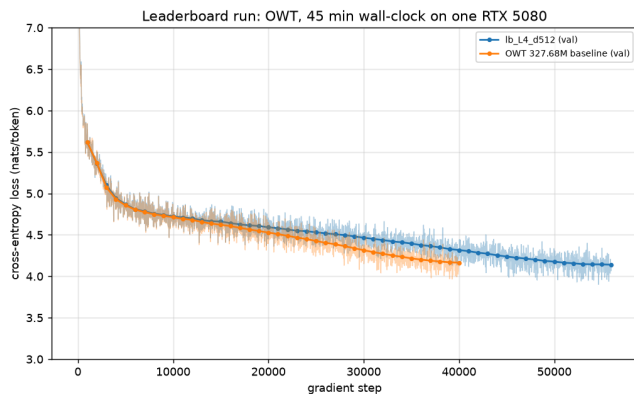
What we did. (1) The 40.96M-token OWT runs (owt40M_lr3e-3/1e-3) fixed the LR at 3e-3 (same optimum as on TinyStories). (2) B=64 with the 32k vocabulary does not fit our memory share (the fp32 logits alone are 2.1 GB per copy), so B=32. (3) A small "equal wall-clock" architecture probe: 4.5 minutes each with the cosine schedule sized from a 100-step throughput measurement so it ends at 4 min (figures/lb_probe_*):



config	non-emb. params	tokens/s	steps in 4 min	val loss @4 min
4 layers, d 512 (baseline)	12.5M	178k	4,875	4.579
8 layers, d 512	24.9M	135k	3,698	4.755
6 layers, d 768 (12 heads, d_ff 2048)	42.5M	111k	3,060	5.311

At this (tiny) compute the smallest model wins clearly, so the leaderboard run keeps the baseline architecture; the deeper/wider models are far from converged after a few thousand steps and lose more from fewer steps than they gain from capacity (a 10× longer run would narrow, and possibly flip, the ranking, but we could not afford to test that at 45 min each). Other candidate modifications (weight tying, QK-norm, a Muon-style optimizer, longer context) were not implemented in the shared model / optimizer code and were left out rather than tested untuned. (4) Final run `leaderboard/lb_L4_d512`: OWT, baseline architecture, B=32, ctx 256, AdamW $\beta=(0.9,0.95)$, wd 0.1, clip 1.0, peak LR 3e-3, warmup 4% (2,238 steps), cosine to 3e-4 ending at step 55,961 (chosen from the measured 177.7k tok/s so that the schedule finishes at ≈ 43 min), `--max-wallclock-min 44.5` as a hard stop, eval every 1,000 steps on 40 fixed batches (327k tokens).

Result. 55,961 steps = 458.5M tokens in **43.1 min wall-clock** (compile + evals included; stopped by `max_steps`, not by the time limit); validation loss **4.140** on the fixed batches, **4.131** on the first 25.6M tokens of the OWT validation set; curve `figures/leaderboard_{steps,wallclock}.png` (wall-clock axis ends at 43 min). It beats the naive 5.0 baseline by a wide margin and the 327.68M-token OWT run (4.165 / 4.157) by only ≈ 0.02 , which is the honest picture at this scale: 40% more tokens buys very little because the loss is decreasing roughly linearly in $\log$ tokens (≈ 0.15 per doubling here) and the small model is the bottleneck. On a B200 the same recipe would see ≈ 2.5 – 3.5 B tokens in 45 minutes (5–8× more) and would sit around 3.7–3.9 by that extrapolation; getting substantially lower requires the model changes above plus a larger model, which is what the top leaderboard entries do.



Experiment log (assignment §7)

One row per training run. All runs: RTX 5080 (shared, ≈ 6 GB VRAM cap, one run at a time), bf16 autocast + TF32 + torch.compile, python -m cs336_basics.train (§5 CLI; the SDPA fp32-softmax fix in model.py is required for compiled bf16 — see section7_experiments.md). Unless the *variant* column says otherwise a run uses the §7.2 baseline: TinyStories 10k vocab, ctx 256, d_model 512, d_ff 1344, 4 layers, 16 heads, RoPE $\Theta=1e4$, AdamW $\beta=(0.9,0.95)$ $\epsilon=1e-8$ wd 0.1, grad-clip 1.0, warmup 5% of steps

- cosine to lr/10 at the last step, batch 32, seed 0. *final val loss* = mean CE on the 40 fixed validation batches (327,680 tokens; --eval-batches scaled with the batch so the token count is the same for every run) at the last step; *min val loss* = best evaluation during the run; *final train loss* = mean of the last 10 logged train losses; *tok/s* = median training throughput (excludes eval time; wall-clock includes it). Full-validation-set numbers for the headline checkpoints (scripts/eval_checkpoint.py, 5,465,856 tokens):

checkpoint	val loss (40 fixed batches)	val loss (full validation set)
ts_full_lr2e-3/ckpt_final.pt (best TinyStories model)	1.3796	1.3909 (ppl 4.02)
ts_full_lr3e-3/ckpt_final.pt	1.3945	1.4066 (ppl 4.08)
lr_sweep/lr3e-3/ckpt_final.pt (40.96M tokens)	1.6485	1.6597 (ppl 5.26)
owt/owt_full_lr3e-3/ckpt_final.pt	4.1651	4.1572 (ppl 63.9; first 25.6M of 66.4M val tokens)
leaderboard/lb_L4_d512/ckpt_final.pt	4.1398	4.1310 (ppl 62.2; first 25.6M val tokens)

Regenerate the table / figures: scripts/plot_runs.py --table <run dirs> and scripts/plot_runs.py --out writeup/figures/<name> <label:run dir> ... (all figure commands: scripts/sec7_make_figures.sh; table assembly: run-workspace make_explog.sh). Run directories (log.jsonl, config.json, stdout.log, checkpoints) live in /home/c12/workspace/claude_workspace/cs336_a1/sec7-exp/runs/<group>/<run> and are not committed (≈ 270 MB per checkpoint).

Figures (each exists as _steps.png and _wallclock.png in writeup/figures/): lr_sweep, lr_divergence, ts_full, batch_size, ablation_rmsnorm, ablation_postnorm, ablation_nope, ablation_swiglu, owt, lb_probe, leaderboard.

Runs

Learning-rate sweep and divergence (§learning_rate) — 40.96M tokens

run	variant	lr	batch	steps	tokens	wall-clock (min)	tok/s	final train loss	final val loss	min val loss	note
lr1e-4	baseline	0.0001	32	5000	40,960,000	2.7	283k	2.287	2.291	2.291	
lr3e-4	baseline	0.0003	32	5000	40,960,000	2.3	335k	1.867	1.876	1.876	
lr1e-3	baseline	0.001	32	5000	40,960,000	2.4	321k	1.660	1.669	1.669	
lr2e-3	baseline	0.002	32	5000	40,960,000	2.6	291k	1.631	1.639	1.639	
lr3e-3	baseline	0.003	32	5000	40,960,000	2.7	289k	1.642	1.648	1.648	
lr5e-3	baseline	0.005	32	5000	40,960,000	2.8	283k	1.720	1.727	1.727	
lr1e-2	baseline	0.01	32	5000	40,960,000	2.7	326k	1.794	1.803	1.803	
lr3e-2	baseline	0.03	32	5000	40,960,000	2.6	323k	2.374	2.359	2.359	
lr1e-1	baseline	0.1	32	5000	40,960,000	2.8	282k	3.091	3.049	3.049	
lr3e-2_noclip	grad_clip=0.0	0.03	32	5000	40,960,000	2.2	343k	2.393	2.378	2.378	
lr1e-1_noclip	grad_clip=0.0	0.1	32	5000	40,960,000	2.2	348k	3.062	3.020	3.020	

Notes: 3e-2 / 1e-1 (with or without clipping) never NaN but the loss blows up to 4.6–5.7 during warmup and only partly recovers (see figures/lr_divergence). Warmup 250 steps for all short runs.

Full-budget TinyStories runs (327.68M tokens) and second seeds

run	variant	lr	batch	steps	tokens	wall-clock (min)	tok/s	final train loss	final val loss	min val loss	note
ts_full_lr3e-3	baseline	0.003	32	40000	327,680,000	17.4	334k	1.367	1.394	1.394	
ts_full_lr2e-3	baseline	0.002	32	40000	327,680,000	17.1	331k	1.355	1.380	1.380	
lr3e-3_seed1	baseline	0.003	32	5000	40,960,000	3.8	158k	1.625	1.634	1.634	
lr2e-3_seed1	baseline	0.002	32	5000	40,960,000	2.2	336k	1.620	1.628	1.628	

Notes: full runs use warmup 2,000, eval every 1,000 steps, checkpoints every 10,000; seed-1 runs repeat lr_sweep/lr3e-3 and lr2e-3 with `--seed 1` (lr3e-3_seed1 overlapped with a validation-set evaluation on the same GPU, hence its lower tok/s / longer wall-clock).

Ablations (§7.3) — 40.96M tokens, B=32

run	variant	lr	batch	steps	tokens	wall-clock (min)	tok/s	final train loss	final val loss	min val loss	note
no_rmsnorm_lr3e-3	no_rmsnorm=True	0.003	32	5000	40,960,000	2.7	319k	4040.024	nan	nan	diverged (NaN at step 220)
no_rmsnorm_lr1e-3	no_rmsnorm=True	0.001	32	5000	40,960,000	2.3	321k	1.681	1.690	1.690	
no_rmsnorm_lr3e-4	no_rmsnorm=True	0.0003	32	5000	40,960,000	2.3	327k	1.875	1.879	1.879	
post_norm_lr3e-3	post_norm=True	0.003	32	5000	40,960,000	2.4	327k	1.695	1.701	1.701	
nope_lr3e-3	no_rope=True	0.003	32	5000	40,960,000	2.3	339k	1.792	1.798	1.798	
silu_dff2048_lr3e-3	d_ff=2048, ffn=silu	0.003	32	5000	40,960,000	2.3	339k	1.626	1.635	1.635	
silu_dff2048_lr3e-3_seed1	d_ff=2048, ffn=silu	0.003	32	5000	40,960,000	2.2	338k	1.639	1.646	1.646	

Baselines for these are lr_sweep/lr3e-3 (1.648), lr1e-3 (1.669), lr3e-4 (1.876) and seeds/lr3e-3_seed1 (1.634).

Batch size (§batch_size_experiment) — 40.96M tokens each, LR re-tuned

run	variant	lr	batch	steps	tokens	wall-clock (min)	tok/s	final train loss	final val loss	min val loss	note
bs1_lr3e-4	baseline	0.0003	1	160000	40,960,000	14.5	47k	2.222	2.099	2.099	
bs1_lr1e-3	baseline	0.001	1	160000	40,960,000	13.9	52k	2.312	2.188	2.188	
bs16_lr2e-3	baseline	0.002	16	10000	40,960,000	2.4	305k	1.678	1.679	1.679	
bs16_lr3e-3	baseline	0.003	16	10000	40,960,000	2.5	304k	1.696	1.695	1.695	
lr2e-3	baseline	0.002	32	5000	40,960,000	2.6	291k	1.631	1.639	1.639	
lr3e-3	baseline	0.003	32	5000	40,960,000	2.7	289k	1.642	1.648	1.648	
bs64_lr3e-3	baseline	0.003	64	2500	40,960,000	2.3	350k	1.631	1.625	1.625	
bs64_lr4e-3	baseline	0.004	64	2500	40,960,000	2.1	351k	1.644	1.639	1.639	

Notes: warmup = 5% of steps; `--eval-batches 1280/B` keeps the evaluated token count constant. B=128 / 256 were not run: peak allocated memory 0.47 / 1.56 / 2.72 / 5.05 GiB for B = 1 / 16 / 32 / 64 and OOM (>8.5 GiB) at B=128 under our ≈6 GB share of the 16 GB card.

OpenWebText (§main_experiment) — 32k vocab, same architecture

run	variant	lr	batch	steps	tokens	wall-clock (min)	tok/s	final train loss	final val loss	min val loss	note
owt40M_lr3e-3	vocab_size=32000, data=OWT	0.003	32	5000	40,960,000	4.2	181k	4.554	4.572	4.572	
owt40M_lr1e-3	vocab_size=32000, data=OWT	0.001	32	5000	40,960,000	4.0	182k	4.692	4.708	4.708	
owt_full_lr3e-3	vocab_size=32000, data=OWT	0.003	32	40000	327,680,000	31.3	178k	4.120	4.165	4.165	

Notes: OWT model has 45.22M parameters (32k-vocab embedding + LM head); full-validation-set loss of owt_full_lr3e-3 = 4.157 over the first 25.6M validation tokens.

Leaderboard probes (equal 4.5-min wall-clock, OWT, B=32, lr 3e-3) and the 45-min run

run	variant	lr	batch	steps	tokens	wall-clock (min)	tok/s	final train loss	final val loss	min val loss	note
L4_d512	vocab_size=32000, data=OWT	0.003	32	4875	39,936,000	4.0	178k	4.576	4.579	4.578	
L8_d512	vocab_size=32000, num_layers=8, data=OWT	0.003	32	3698	30,294,016	4.1	134k	4.752	4.755	4.755	
L6_d768	vocab_size=32000, d_model=768, num_layers=6, num_heads=12, d_ff=2048, data=OWT	0.003	32	3060	25,067,520	4.0	112k	5.265	5.311	5.311	
lb_L4_d512	vocab_size=32000, data=OWT	0.003	32	55961	458,432,512	43.1	181k	4.108	4.140	4.140	

Notes: probe step counts (4,875 / 3,698 / 3,060) were set from a 100-step throughput measurement so that each cosine schedule ends at ≈ 4 min; the leaderboard run's step count (55,961) was set the same way for 43 min, with a hard `--max-wallclock-min 44.5 stop`.

Not in the tables

Smoke / debugging runs (`smoke_tput`, `dbg_*`: throughput measurement and the `compile+bf16 NaN` bisection on TinyStories-valid as training data), the 100-step `lb_probe/tput_*` throughput probes, and a VRAM probe (`vram_probe.py`).